Data Engineering

Project 2

VLM-Based Natural Language Long-Tail Fashion Attribute Retrieval System

Team 1

김민영 남재은 박시연 임서정 서지민 손시영 정지영

Contents

Project Overview

Topic Introduction

Limit 1

"Loose lavender knit long sleeve"
Cannot recognize color + fit + material combination

Limit 2

"White vertical stripe long one-piece"
No simultaneous filter for pattern + color + length

Limit 3

"High-waist wide-leg denim pants"
No search support for bottom attributes (waist rise / fit)

Our Approach

One-liner: **Natural language-based long-tail fashion attribute search system**

VLM-generated captions + structured attribute tags → semantic vector search

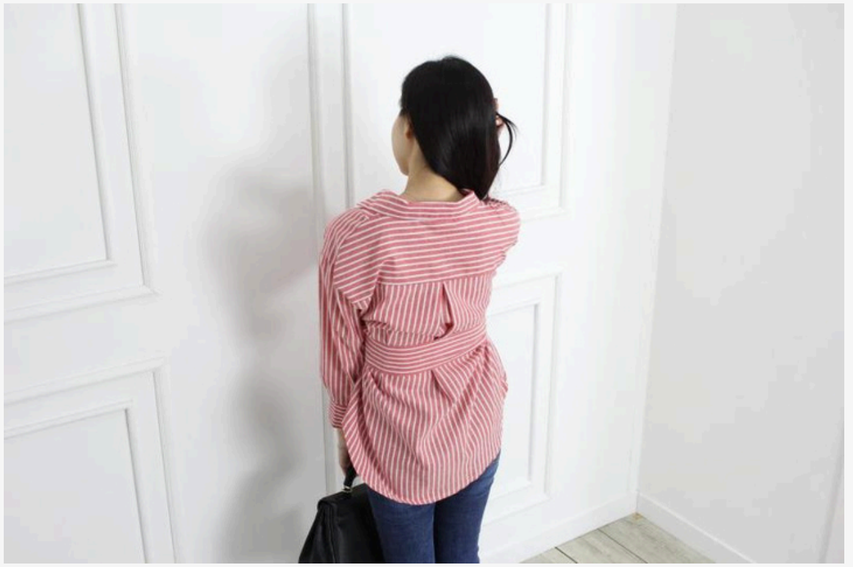
Visual Embedding / VLM Captioning / Attribute Classifier

Dataset Introduction

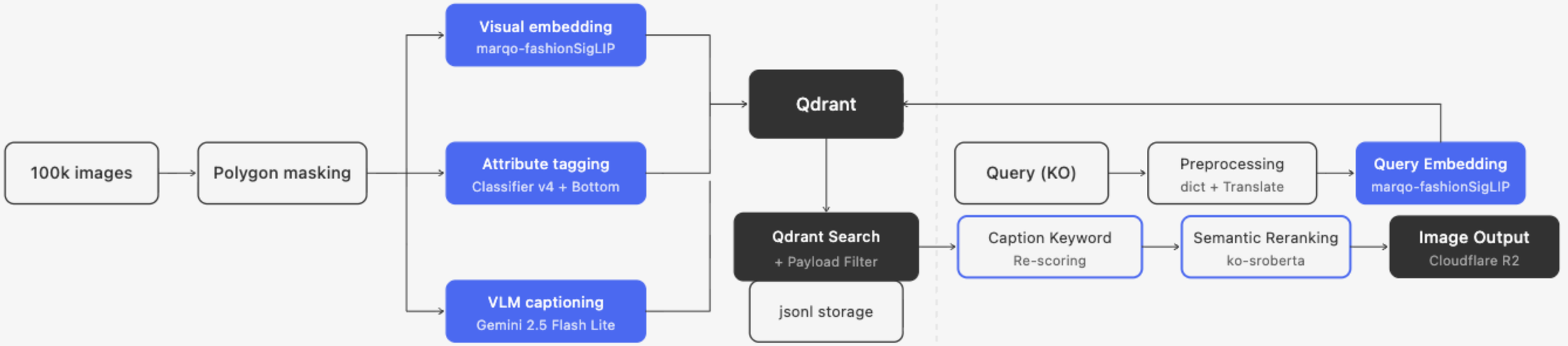
Title: Dataset — K-Fashion (AI Hub)

1,200,000	23	186
images	categories	attributes

- Subset used: **100,000 images** (wear type only, resolution ≥ 224px, deduplication applied)
- Provided: **Polygon masking** coordinates / Bounding box / Style & attribute labels
- Usage: Polygon → masking image generation → VLM captioning / classifier training



System Pipeline



Data Collect & Storage

Data Storage with Cloudflare R2

Cloudflare R2 Object Storage

- Storage for image and labeling JSON data
- Cloud-based centralized repository

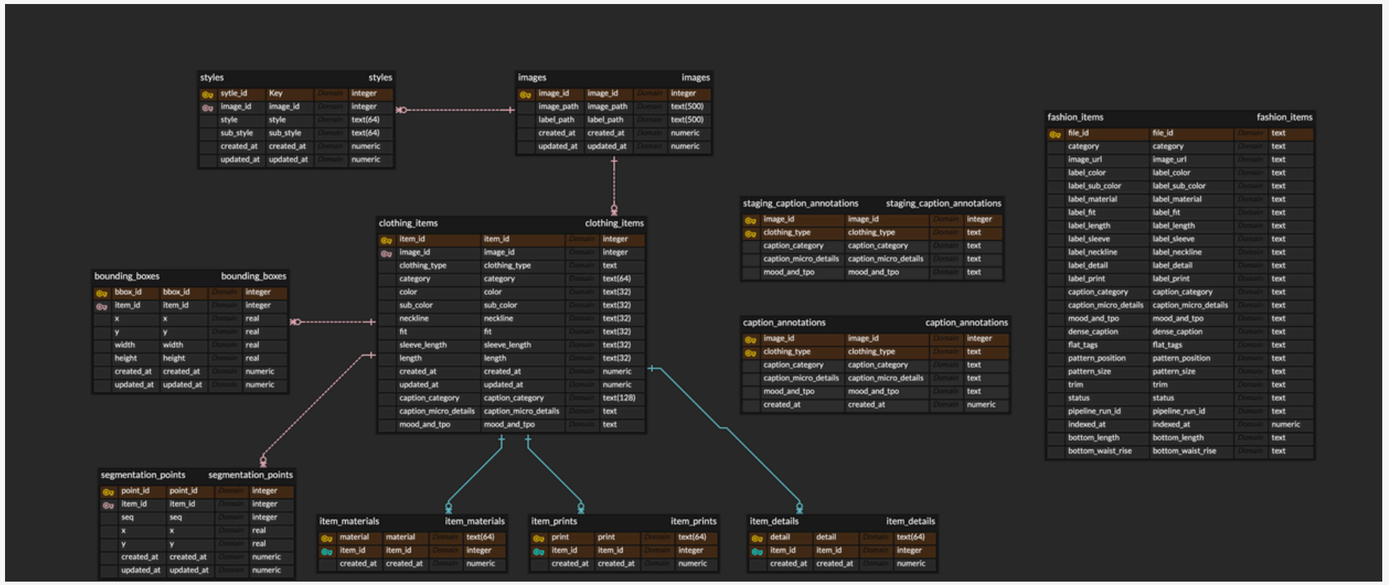
Problem Encountered

- Korean folder names in original dataset
- Path recognition errors in the annotation tool

Solution

- Created new English-named folders
- Replicated all image and JSON labeling data into them

Relational Database (RDB)



[ERD of Relational Database Schema]

Existing Metadata Storage

- Fits the relational model perfectly

Structured Pipeline Outputs

- Uses SQLite schema to structurally store future creating data.

Cloud-Based Collaboration

- Powered by Turso for cloud hosting and multi-user concurrent access.

Vector Database with Qdrant

01. User Query

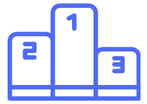
Natural language Input

02. Embedding

Text Encoder
→ 768-D Vector

03. Qdrant Search

Cosine Similirarity Search



04. Top-K Results

Visual Top-K
Candidates Returned

COLLECTION INFO

Qdrant Collection Specification

collection	model	dimension	distance metric	stored vectors	environment
fashion_visual_index	marqo-fashionSigLIP	768	Cosine Similarity	156,713	Qdrant Local

Data Annotation & Model Training

Data Annotation Process

01

Label Schema Definition

02

Manual Annotation

6,000 images

03

Model Training and Automated Annotation

100,000 images

04

Feedback Process

Label Schema Definition

1. **Pattern position** : allover, front, back, sleeve, shoulder, hem, neckline, side, none
2. **Pattern size** : tiny, medium, large, none
3. **Hem finish** : standard, raw, rolled, banded, mixed
4. **Waist rise** : high_rise, mid_rise, unknown
5. **Bottom length** : micro_short, short, bermuda, capri, midi, ankle, floor_maxi, unknown

- Attributes were selected based on their stylistic importance and frequent usage in fashion e-commerce search queries.
- Used annotation guidelines to maintain labeling consistency.
- ex) Pattern position definition

옵션	정의	판단 기준
allover	옷의 거의 모든 면에 패턴이 균일하게 분포	옷 면적의 70% 이상에 비슷한 밀도로 분포할 때
front	앞판 (가슴~배 영역)	앞면의 중앙 부위에 패턴이 위치
back	뒷판	뒷면이 보이는 사진에서 뒷면에 패턴이 위치
sleeve	소매	팔 부위에 패턴이 있을 때 (한쪽만 있어도 sleeve 선택)
shoulder	어깨/요크 절개선 부근	어깨선~가슴 위쪽 영역
hem	밑단	옷의 가장 아래쪽 끝부분
neckline	목 주변	목둘레선과 그 인접 부위 (칼라 자체도 포함)
side	옆면	바지 옆면, 상의 옆구리 부분
none	패턴 없음	무지 옷 (단색이거나 텍스처만 있고 패턴 없음)

Manual Annotation – Tool Used

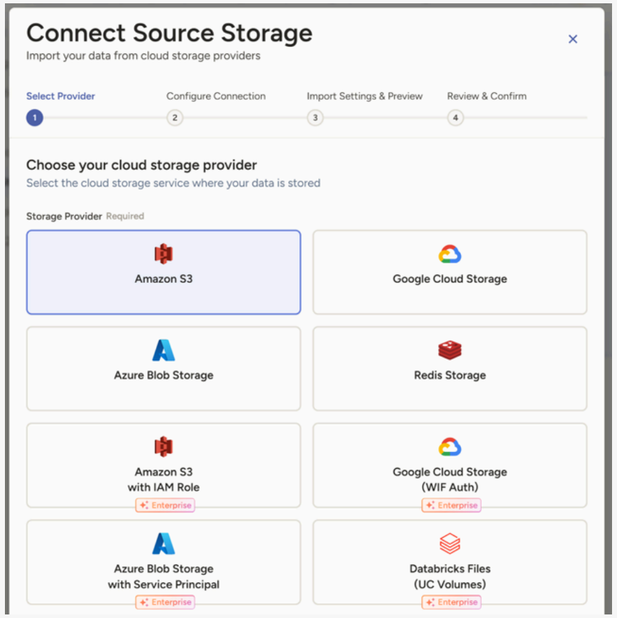
- **Tool used : Label Studio**

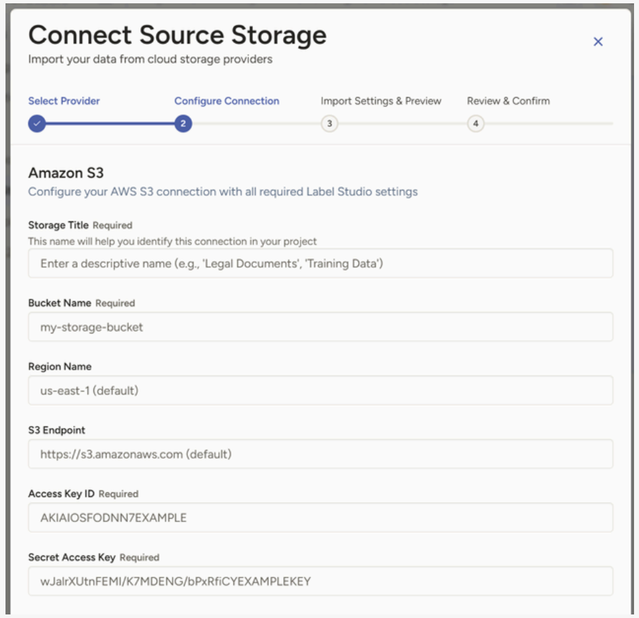
- **Reasons for Selection**
  - Collaboration Features Support
  - Free and Open Source
  - Provides a Multi-Attribute Annotation Interface
  - Customizable Interface Tailored to Project Requirements

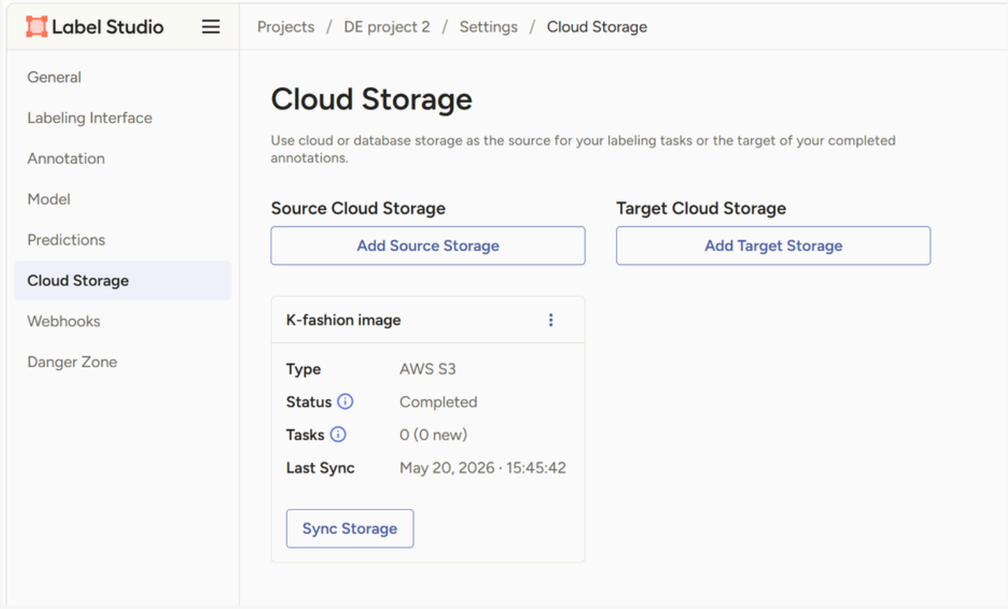
Manual Annotation – Environment Setup

1. Add Cloud Storage

- Go to project settings in Label Studio and select 'Cloud Storage'
- Enter the cloud storage access key



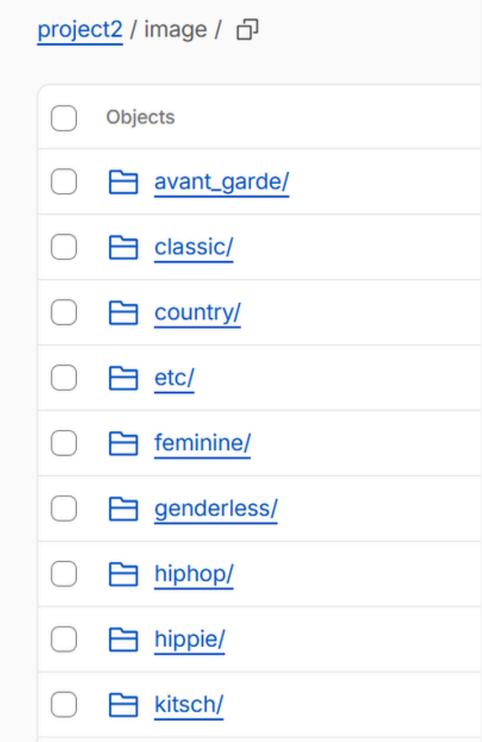


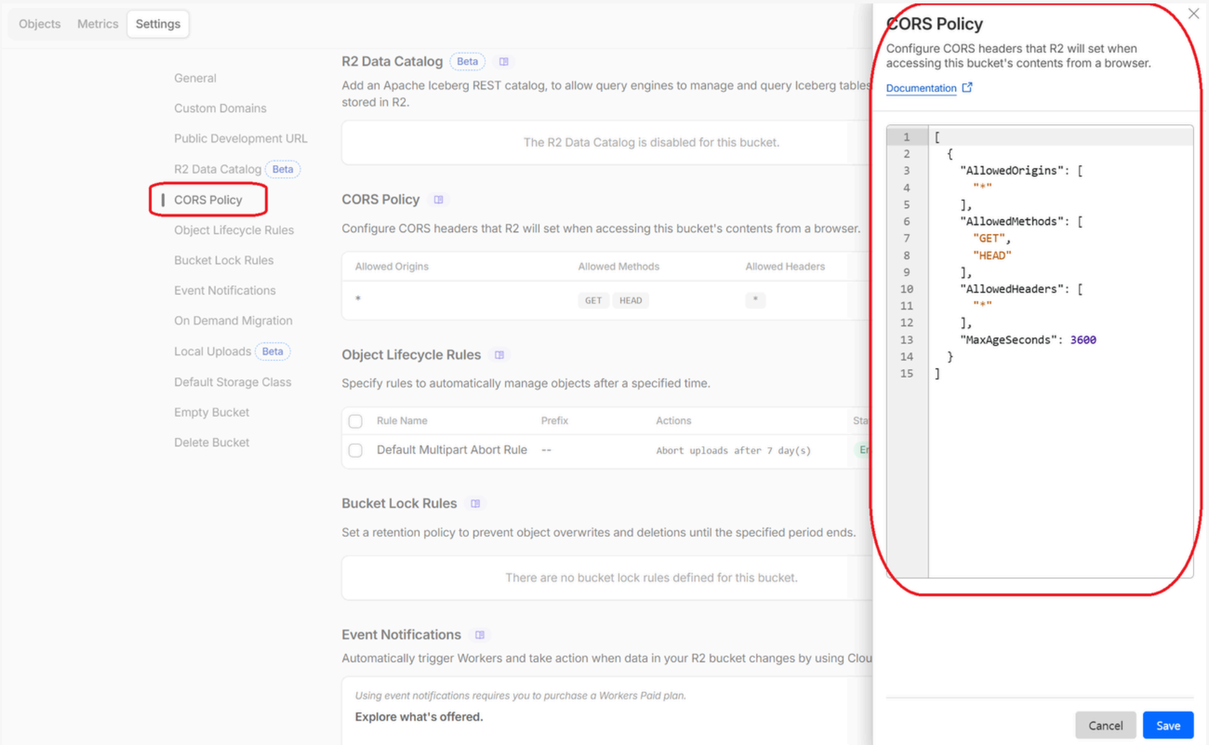


Manual Annotation – Environment Setup

2. Cloud Storage Sync

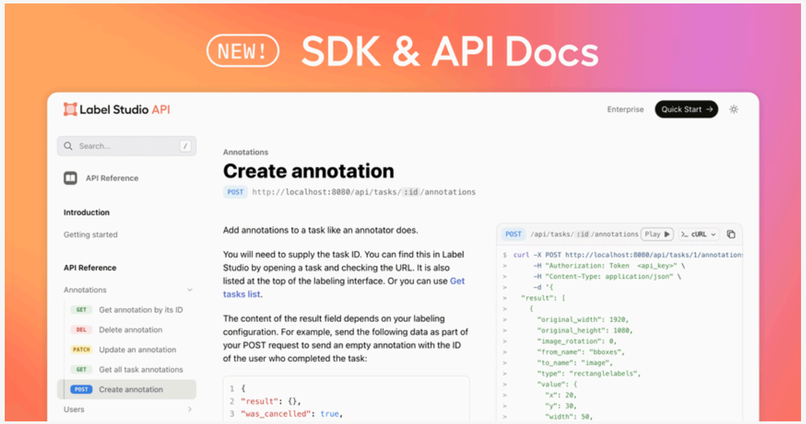
- Rename folder names in English
- Configure CORS Policies for access control





Manual Annotation – Environment Setup

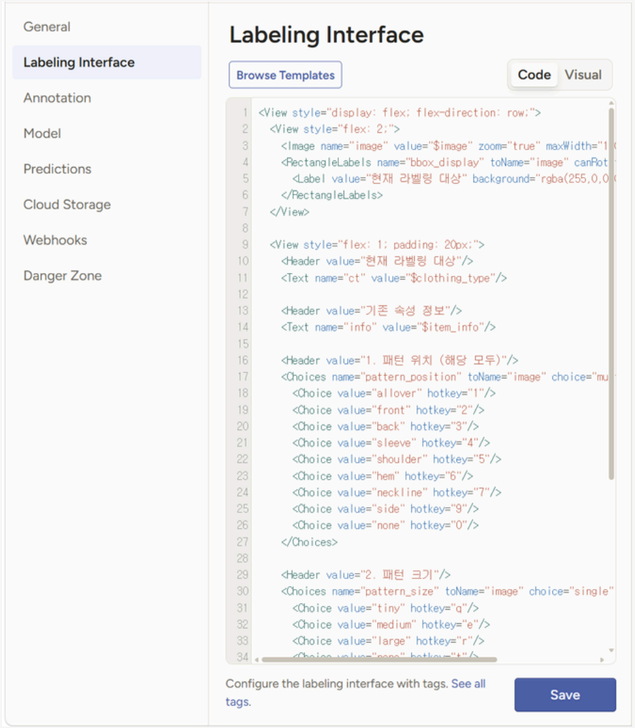
3. **Tasks Generation**
   - Library Used: Boto3, Label Studio SDK
   - Process
     i. Sample items for annotation using the .json labeling files provided with the original dataset
     ii. Generate task JSON files containing image URLs
     iii. Ensure that Label Studio has access to the images stored in the R2 bucket. Public Access must be allowed in R2 bucket settings
     iv. Create annotation tasks in Label Studio

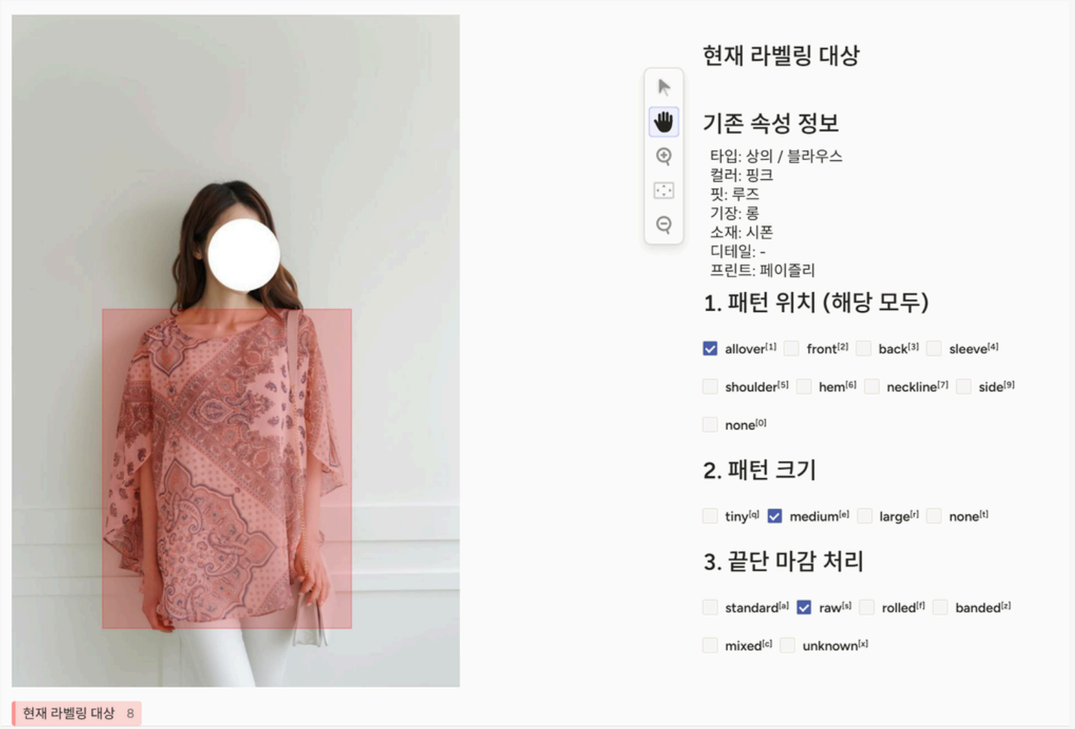


Manual Annotation – Environment Setup

4. Annotation Interface Setup

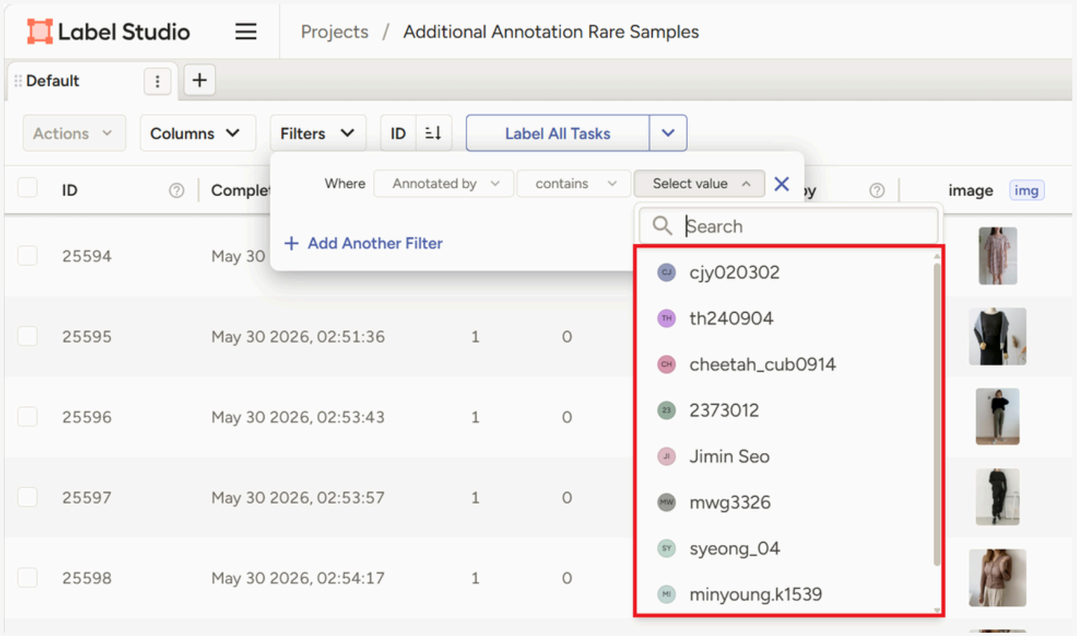
- Go to project setting and select ‘Labeling Interface > Code’
- Display the target image along with the corresponding item bounding box
- Select the checkbox for each of the three attributes to be annotated





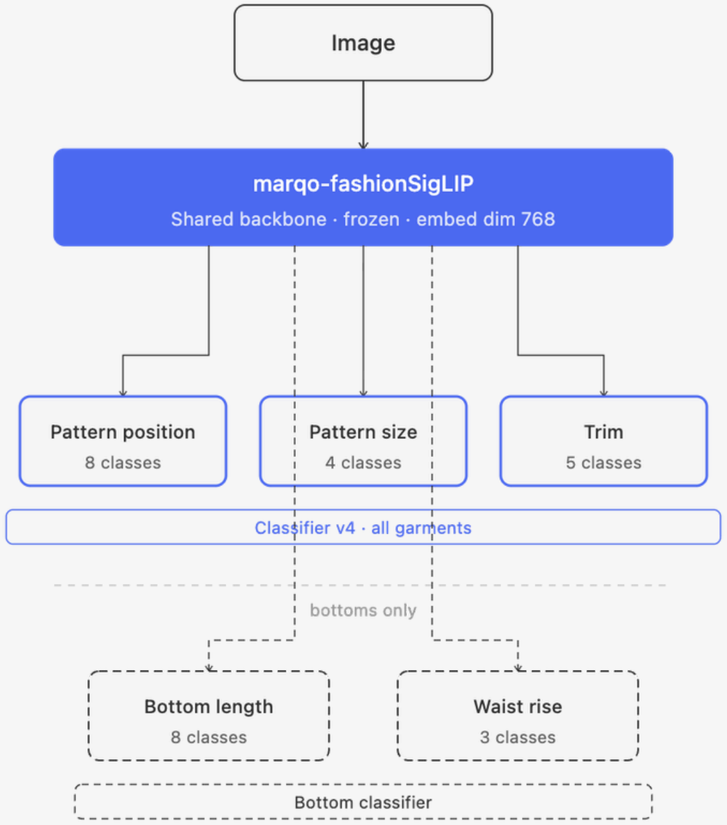
Manual Annotation – Environment Setup

5. **Team Collaboration**
   - Tool Used : Ngrok
   - Used a team member's laptop as the Label Studio server
   - Used Ngrok to provide secure external access for other team members



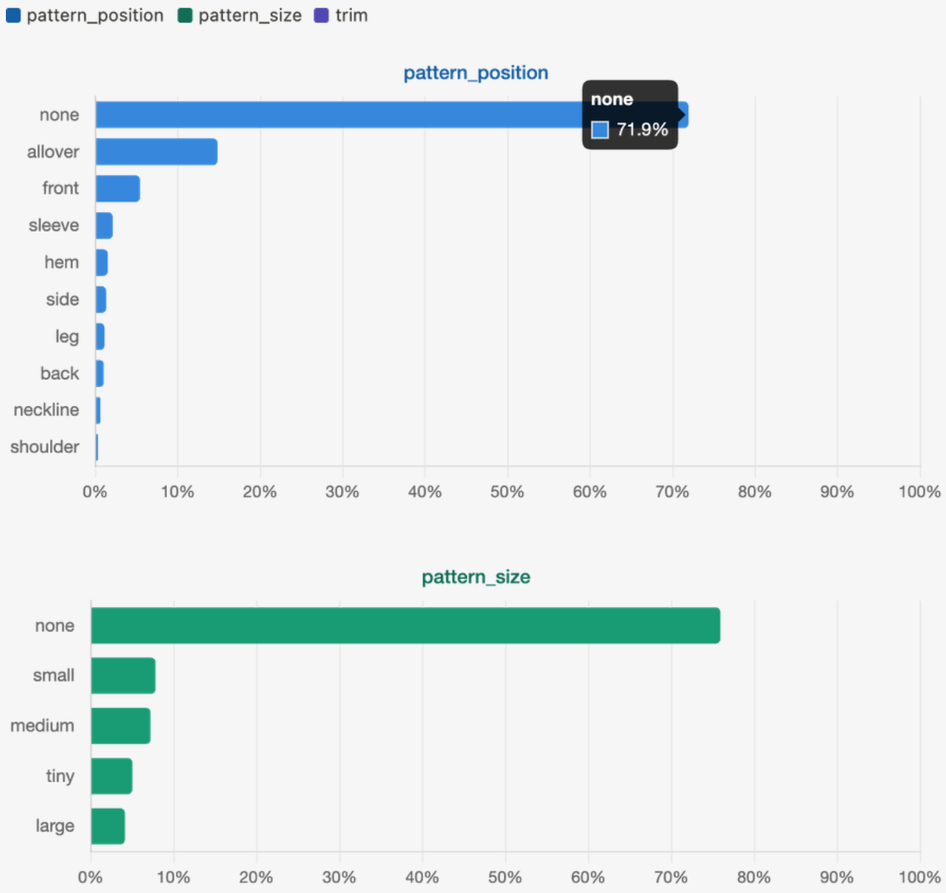
Model Structure

- Image
- └── **marqo-fashionSigLIP** (shared backbone, frozen)
- ├── Pattern Position Head → multi-label
- ├── Pattern Size Head → 4 classes
- ├── Trim Head → 5 classes
- ├── Bottom Length Head → 8 classes (bottom only)
- └── Bottom Waist Rise Head → 3 classes (bottom only)

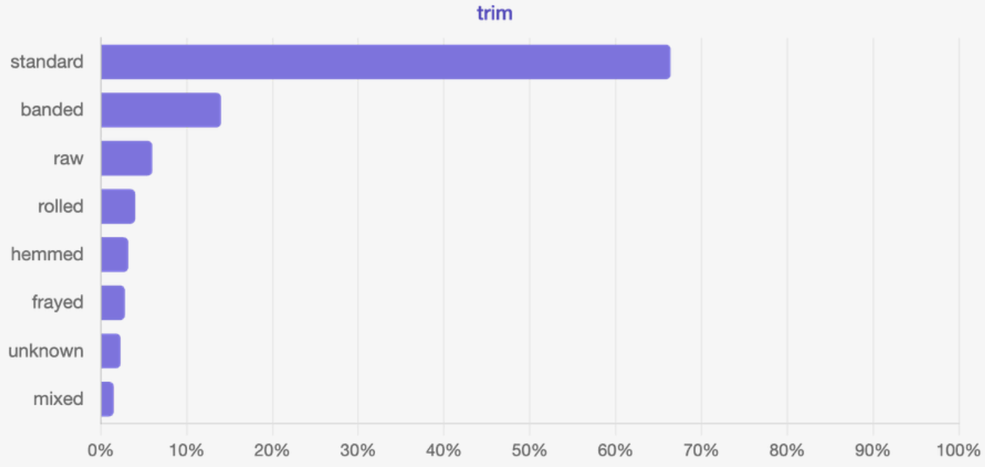


- Backbone: marqo-fashionSigLIP (ViT-B-16, embed_dim 768)
- Backbone Frozen + **classifier head training**
- Top / Outerwear / One-piece → Classifier v4 (pp / ps / trim joint training)
- Bottom → Bottom Classifier (length / waist joint training)

Training Strategy



Objective	Solution
Class imbalance handling	log-scale class weights
Majority class undersampling	removed in final version (full data utilized)
Minority class oversampling	augmentation applied
Early Stopping (patience=5)	
Data quality control	Followed strong Annotation Guideline



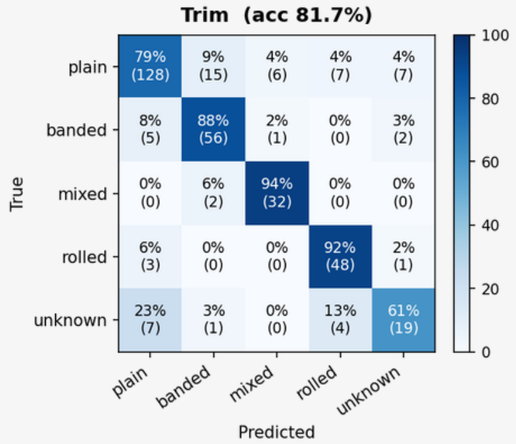
Classifier Final Performance

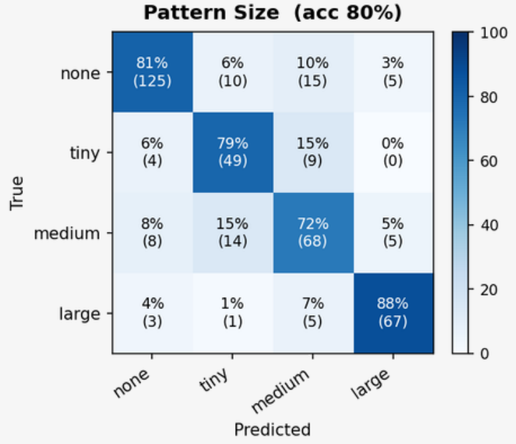
Final Performance

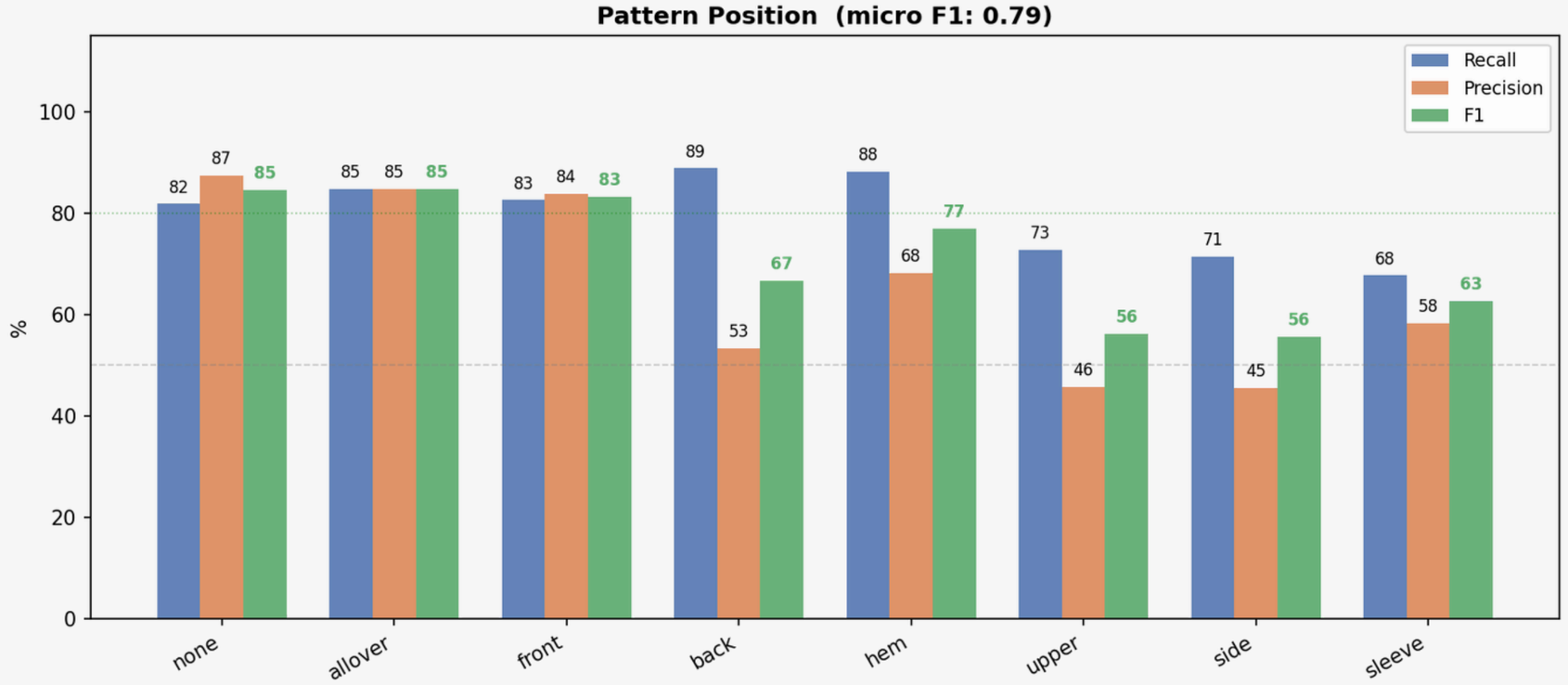
분류기	지표	값
Pattern Position	Micro F1	0.79
Pattern Size	accuracy	80%
Trim	accuracy	81.70%
Bottom Lehgth	accuracy	73%
Bottom waist Rise	accuracy	77%

Classifier Final Performance

Confusion Matrix

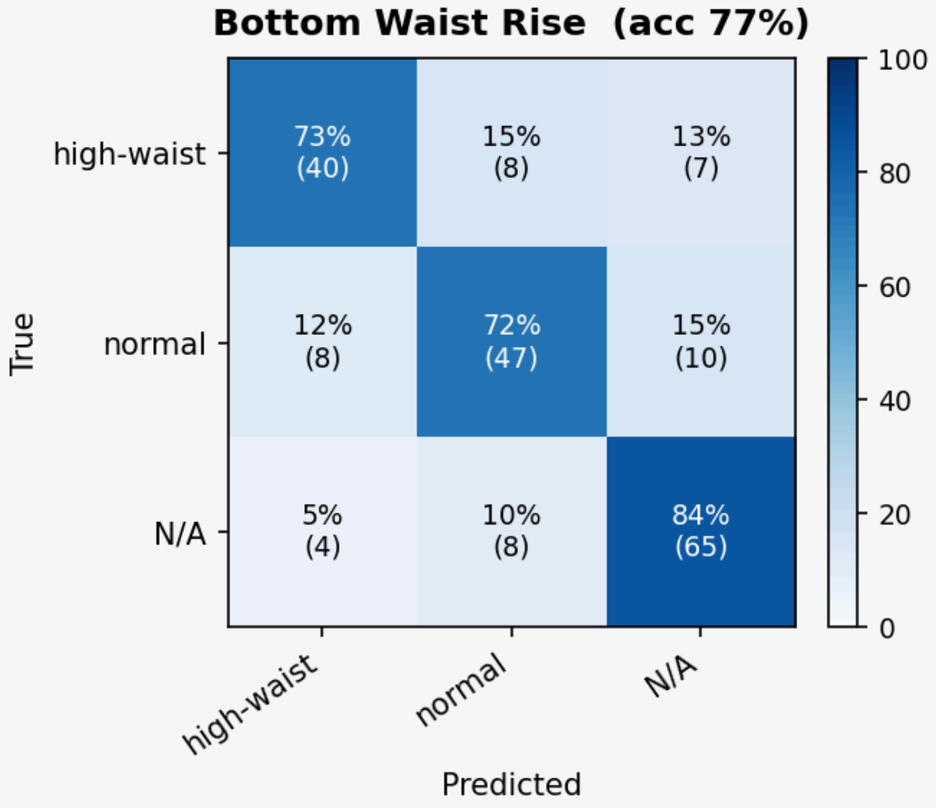


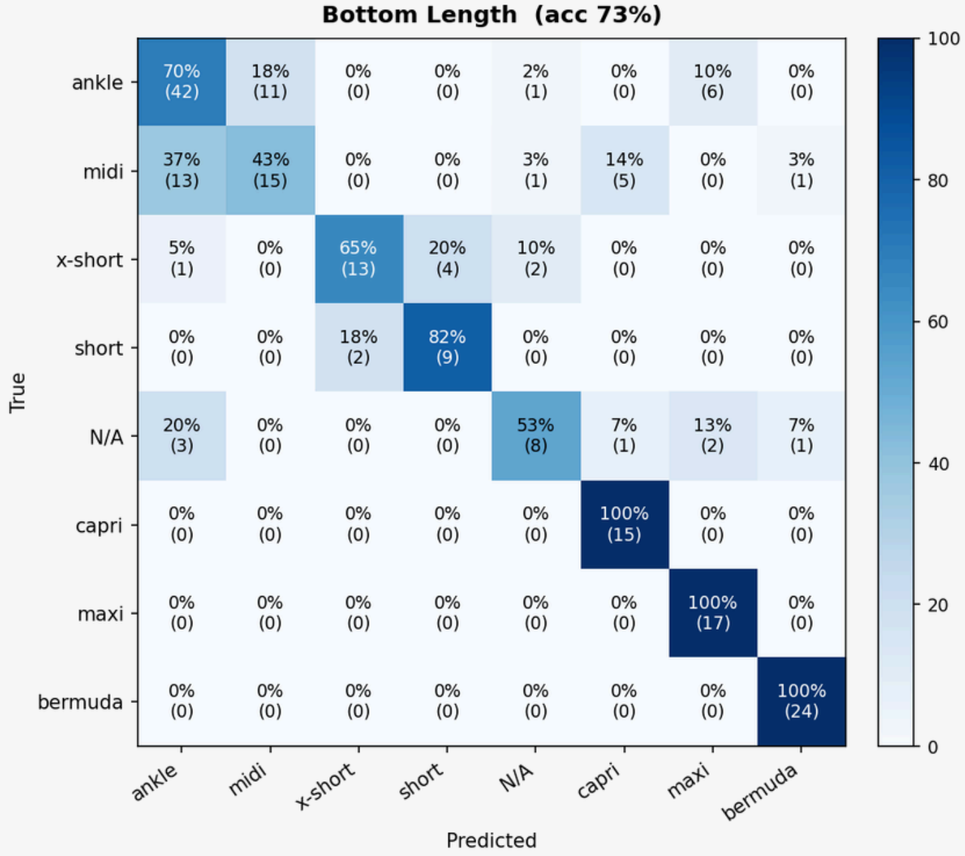




Classifier Final Performance

Confusion Matrix





Feedback Process

Stage 1

- Randomly sampled **1,500** images
- Limited annotation to 3 annotators to maintain labeling consistency
- **Issue**: Severe class imbalance was observed

Stage 2

- Trained a model on the 100,000-image dataset and randomly sampled **1,500** images which were predicted as minority classes
- Performed by the same 3 annotators from Stage 1

Stage 3

- In lab session, TA suggested that manually annotating approximately 5% of the dataset could be beneficial
- Before completing the Stage 2, the team decided to conduct an additional **2,000** manual annotations
- To accelerate progress, all 7 members participated in the annotation

Stage 4

- Added two new annotation labels: Waist Rise and Bottom Length.
- Randomly sampled **750** images and performed manual annotation by 2 annotators

Stage 5

- To further improve model performance from Stage 4, additional manual annotation was conducted on samples with low prediction confidence
- Annotated **250** images by 2 annotators from Stage 4

VLM Captioning

VLM Captioning

Objective

- To detect minor characteristics on each product that are not in labels.
  - caption_micro_details
  - mood_and_tpo

VLM Captioning

1. **Image Data Polygon Masking**
   - Before caption generation, all images were masked by item category using the provided polygon annotations
   - Item categories: Outerwear, Top, Bottom, and Dress
   - Final output: 156,713 masked JPG images (6.59 GB)

VLM Captioning

2. Prompt Engineering

- For the best performance, extensive experiments were conducted
- Models Tested : **Gemini 2.5 Flash Lite**, Gemini 2.5 Flash, GPT-4o-mini, Qwen2.5-VL-3B
- Implemented asynchronous parallel processing and structured output enforcement for throughput and reliability
- **Final Promts:**

```python
def _build_contents(img_bytes: bytes, mime_type: str, label_hint: str | None) -> list:
    contents = [
        types.Part.from_bytes(data=img_bytes, mime_type=mime_type),
        SYSTEM_PROMPT,
    ]
    if label_hint:
        contents.append(
            f"[참고: 이미지 사전 확보 정보]\n{label_hint}\n"
            "위 대분류/카테고리와 일관된 서브카테고리를 생성하십시오. "
            "대분류·카테고리 자체는 micro_details에 포함하지 마십시오."
        )
    return contents
```

```
Role:
당신은 한국 이커머스 패션 플랫폼의 전문 카탈로그 데이터 구축 모델입니다.

Input Context:
입력 이미지는 의류 실루엣만 남기고 배경을 순백색(RGB 255,255,255)으로 마스킹 처리한 이미지입니다.
흰색 영역은 제거된 배경이므로 완전히 무시하고, 의류 객체만 분석하십시오.

Task:
아래 3개 필드만 추출하십시오.
색상·소재·핏·기장·소매기장·넥라인·프린트는 이미 별도 구조화 데이터로 확보되어 있습니다. 절대 출력하지 마십시오.
모든 값은 한국어 명사(구)만 사용하십시오. 완성 문장 금지.

--- 필드 1: category ---
표준 상위 카테고리("팬츠", "티셔츠")보다 세밀한 서브카테고리로 작성하십시오.
실루엣·소재 등 아이템을 특정짓는 특성을 앞에 붙이십시오.

금지 prefix: 색상(화이트·베이지), 핏(루즈핏·오버핏·스키니), 넥라인(브이넥·라운드넥·터틀넥)
→ 이 정보들은 별도 데이터로 이미 확보되어 있습니다.

프린트 패턴(스트라이프·체크·플로럴)은 아이템 정체성의 핵심인 경우 허용합니다.
  허용: 체크 롱원피스, 스트라이프 블라우스, 플로럴 미디스커트
  불필요: 무지 티셔츠 → 그냥 티셔츠

좋은 예:
  스트레이트 데님팬츠, 와이드 린넨팬츠, 카고팬츠, 플리츠 미디스커트,
  셔링 미디원피스, 셔츠원피스, 체크 롱원피스, 트위드 블레이저, 숏 블레이저,
  니트 가디건, 오버핏 후드티, 우븐 블라우스, 스트라이프 블라우스

나쁜 예:
  루즈핏 셔츠원피스 (X) → 셔츠원피스 (O)
  화이트 블라우스 (X) → 우븐 블라우스 (O)
  브이넥 티셔츠 (X) → 크롭 티셔츠 (O)
```

```
--- Few-shot 예시 ---

입력: 인디고 데님 팬츠 (생지, 화이트 스티치, 컷오프 헴라인, 코퍼 리벳)
출력:
{
  "category": "스트레이트 데님팬츠",
  "micro_details": ["생지", "화이트 스티치", "컷오프 헴라인", "코퍼 리벳"],
  "mood_and_tpo": ["고프코어", "스트리트", "데일리룩"]
}

입력: 베이지 트위드 블레이저 (퍼프 소매, 노치드 라펠, 플랩 포켓, 금장 버튼 3개)
출력:
{
  "category": "트위드 블레이저",
  "micro_details": ["노치드 라펠", "퍼프 소매", "플랩 포켓", "금장 버튼"],
  "mood_and_tpo": ["오피스룩", "하객룩", "페미닌", "클래식"]
}

입력: 화이트 코튼 티셔츠 (단순 디자인, 특별한 디테일 없음)
출력:
{
  "category": "베이직 라운드넥 티셔츠",
  "micro_details": [],
  "mood_and_tpo": ["데일리룩", "이지웨어", "미니멀룩"]
}
```

VLM Captioning

3. **Generated Captions**
   - Final Caption examples

{"file_id": "110102", "category": "top",
"caption_category": "우븐 블라우스",
"caption_micro_details": ["셔링", "퍼프 소매"],
"mood_and_tpo": ["오피스룩", "데이트룩", "페미닌", "클래식"]}

{"file_id": "1101311", "category": "outerwear",
"caption_category": "퀼팅 롱패딩",
"caption_micro_details": ["퀼팅", "드로우코드"],
"mood_and_tpo": ["데일리룩", "이지웨어"]}

Embedding & Retrieval Pipeline

Image & Text Embedding

Pipeline Overview

[Flow Diagram]

Masked Images → Image Embedding → Qdrant Visual Index

VLM Captions → Text Embedding → Experimented & Rejected

→ Caption Keyword Filtering (Adopted)

Image Embedding

Model Selection

Model selection process

FashionCLIP — FAILED
Lower performance vs marqo-fashionSigLIP

↓

marqo-fashionSigLIP — ADOPTED ✓
+36% Recall@10 vs FashionCLIP
768-dim vectors
156,713 images embedded
Input: background-removed masked images

Key stats

156,713	Images Embedded
768	Vector Dimensions
+36%	Recall@10 vs FashionCLIP
~6hr	Processing Time (CPU)

Image Embedding

Model Selection

1124800_top	2026-05-30 오후 9:14
1124810_top	2026-05-30 오후 9:14
1124818_bottom	2026-05-30 오후 9:14
1124818_top	2026-05-30 오후 9:14
1124883_top	
1124890_bottom	
1124890_top	
1124910_dress	
1124922_bottom	
1124922_top	

masking data (images)

DB

```python
# 임베딩 생성
embeddings = {}
for i, filename in enumerate(image_files):
    try:
        img = Image.open(f"{image_dir}/{filename}").convert('RGB')
        image = preprocess_val(img).unsqueeze(0)
        with torch.no_grad():
            emb = model.encode_image(image, normalize=True)
        embeddings[filename] = emb.cpu().numpy()

        if (i+1) % 100 == 0:
            print(f"{i+1}/{len(image_files)} 완료")
            np.save('embeddings_checkpoint.npy', embeddings)
    except Exception as e:
        print(f"에러: {filename} - {e}")
```

qdrant_storage
captions_full_lite.jsonl
image_embeddings_final.npy

Text Embedding

Two Models Tested

✓ first attempt

multilingual-e5-large

•Multilingual support (Korean ✓)
•1,024-dim vectors
•Requires 'passage:' prefix
→ **performance issue**
•**Color context misunderstood (e.g. 'white blouse' → blouse with white stitching)**

NO Prefix

✓ second attempt

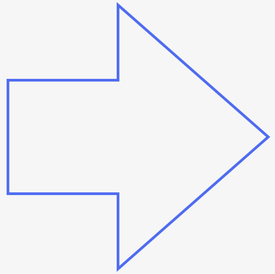
marqo Text Encoder

•Same model (image embedding)
•768-dim vectors
•Results -> visual search
•RRF fusion adds no meaningful improvement

Text Embedding

```
model, _, _ = open_clip.create_model_and_transforms('hf-hub:Marqo/marqo-fashionSigLIP')
tokenizer = open_clip.get_tokenizer('hf-hub:Marqo/marqo-fashionSigLIP')
model = model.cuda().eval()
```

```
for i in range(0, len(texts), batch_size):
    batch = texts[i:i+batch_size]
    ids = [b[0] for b in batch]
    txts = [b[1] for b in batch]
    tokens = tokenizer(txts).cuda()
    with torch.no_grad(), torch.amp.autocast('cuda'):
        embs = model.encode_text(tokens, normalize=True)
    for file_id, emb in zip(ids, embs):
        text_embeddings_marqo[file_id] = emb.cpu().numpy()
    if (i+batch_size) % 5000 == 0:
        print(f"{i+batch_size}개 완료")
        np.save('text_embeddings_marqo_v2_checkpoint.npy', text_embeddings_marqo)
```



How It Was Created

1. Turso DB query
   clothing_items → caption_category
   + micro_details + mood_and_tpo
2. Text concatenation
   '와이드 팬츠 데일리룩 이지웨어'
3. marqo-fashionSigLIP
   text encoder → 768-dim vector
4. Saved as .npy
   batch_size=64, normalize=True

Approaches Tried

Semantic RRF(e5, marqo) → Rejected

- Combines visual search results with text embedding search results from E5 and Marqo using RRF
- e5 fails to understand color context, same model issue(marqo), caption quality dependency, file_id mapping issue.

Semantic + Caption Filtering → Rejected

- Tried caption filtering after visual + e5/marqo RRF
- Applies boost/penalty by directly matching caption text against visual candidates
- RRF adds noise, lowering performance below caption filtering alone (visual + caption alone: 0.950, with RRF: 0.863, 0.625)

Semantic Threshold Filtering → Rejected

- Filters visual Top-K candidates by text embedding similarity above a threshold
- Optimal threshold varies per query, making it unreliable

Caption Filtering → Adopted

- visual + Caption Keyword Filtering
- Full exclusion removes even correctly retrieved results
- Changed to boost/penalty scoring → confirmed improvement

Issues & Solutions

Translation Correction

- Google Translate post-processing → Failed
- DeepL → worse translation for fashion terms
- Confirmed fashion_dict pre-substitution approach

Material vs Pattern Confusion

- "Vertical stripe" search returns wrinkled dresses in top results
- Tried reinforcing fashion_dict → failed to resolve fundamentally
- Added conflict_pairs → ×0.5 penalty when only texture found for pattern query

Length Classification

- Failed to distinguish "midi skirt" vs "long skirt"
- Adding length group caused performance drop for sweatshirt queries
- Resolved by applying only to bottom/dress, skipping top

Reranker Comparison

- Compared ko-sroberta(Bi-Encoder) vs jina-reranker-v2(Cross-Encoder)
- Top-10 results nearly identical, only order slightly differs
- Selected ko-sroberta for its speed and Korean specialization

Implementation

- **Qdrant Connection**

- **Model Load**
  - marqo-fashionSigLIP — Fashion-domain-specific cross-modal model
    Converts text queries into the same vector space as images for image search
  - ko-sroberta-multitask — Korean-specialized sentence embedding model
    Used for semantic similarity calculation between query and caption (reranking)

Implementation

Dictionaries

- fashion_dict : Fashion terms mistranslated by the translator
  e.g. “비침없는” → “opaque”
  “올풀림“ → “frayed”

- attribute_groups : Groups fashion attributes for caption keyword matching
  "pattern": ["striped", "checked", "floral", ...]
  "texture": ["ribbed", "ruffled", "pleated", ...]
  "material":["denim", "chiffon", "linen", ...]

- conflict_pairs : Prevents confusion between material and pattern

- category_keywords : Used to extract category from query
  "knitwear" → top, "skirt" → bottom, "dress" → dress

Implementation

Function Definitions

- translate_to_english : Replaces fashion terms using fashion_dict, translates the query with Google Translate, and adds a prefix so that the Marqo model recognizes it as an optimized fashion image search query

- encode_text_clip : Converts the translated query into a 768-dimensional vector using the marqo-fashionSigLIP text encoder

- extract_category : Iterates through category_keywords to extract the category (top, bottom, dress, or outerwear) from the query; returns None if no category is found

```python
def translate_to_english(query):
    q = query
    for ko, en in fashion_dict.items():
        q = q.replace(ko, en)
    translated = GoogleTranslator(source='ko', target='en').translate(q)
    fashion_query = f"fashion product photo of {translated}"
    print(f"번역: {query} → {fashion_query}")
    return fashion_query
```

```python
def encode_text_clip(query):
    query_en = translate_to_english(query)
    text_input = clip_tokenizer([query_en])
    with torch.no_grad():
        emb = model.encode_text(text_input)
    emb = emb.squeeze()
    emb = emb / emb.norm()
    return emb.numpy()
```

```python
def extract_category(query):
    for cat, keywords in category_keywords.items():
        for kw in keywords:
            if kw in query:
                return cat
    return None
```

Implementation

Function Definitions

- extract_query_attrs : Extracts three types of information from the query: attributes based on attribute_groups such as pattern, texture, and material, as well as negative and positive expressions

- build_qdrant_filter : Creates a Qdrant query_filter, allowing Qdrant to apply index-based filtering using category, must_not, and must_have conditions. To enable efficient text search, a Full-text index was created on the caption field and a Keyword index on the category field in advance.

```python
def extract_query_attrs(query):
    query_attrs, must_not, must_have = {}, {}, {}
    for phrase, (group, kws) in negation_map.items():
        if phrase in query: must_not.setdefault(group, []).extend(kws)
    for phrase, (group, kws) in positive_map.items():
        if phrase in query: must_have.setdefault(group, []).extend(kws)
    for group, keywords in attribute_groups.items():
        for kw in keywords:
            if kw in query: query_attrs.setdefault(group, []).append(kw)
    return query_attrs, must_not, must_have
```

```python
def build_qdrant_filter(query):

    _, must_not, must_have = extract_query_attrs(query)
    target_cat = extract_category(query)

    must_conditions = []
    must_not_conditions = []

    if target_cat:
        must_conditions.append(
            FieldCondition(key="category", match=MatchValue(value=target_cat))
        )

    for group, kws in must_not.items():
        for kw in kws:
            must_not_conditions.append(
                FieldCondition(key="caption", match=MatchText(text=kw))
            )

    for group, kws in must_have.items():
        for kw in kws:
            must_conditions.append(
                FieldCondition(key="caption", match=MatchText(text=kw))
            )

    if not must_conditions and not must_not_conditions:
        return None

    return Filter(
        must=must_conditions if must_conditions else None,
        must_not=must_not_conditions if must_not_conditions else None,
    )
```

Implementation

Function Definitions

- _caption_filter_logic : Applies boosts and penalties based on attribute_groups and checks conflicts using conflict_pairs

  Penalty if caption and query contradict within a group, boost if they match
  e.g., query: “striped dress”, caption: “checked dress”
  Since “striped” and “checked” conflict within the pattern group, a penalty is applied

  Penalty when the query contains a pattern but the caption contains only a texture
  e.g., query: “striped dress”, caption: “ribbed dress”
  A penalty is applied because ribbed textures may be incorrectly recognized as stripes

```python
def _caption_filter_logic(caption, query_attrs, penalty, boost_val, file_cat=None):

    boost = 1.0
    for group, kws in query_attrs.items():
        if group == "length" and file_cat not in ["bottom", "dress"]:
            continue
        caption_has = [k for k in attribute_groups[group] if k in caption]
        if caption_has:
            boost *= boost_val if any(k in caption for k in kws) else penalty
    for g1, g2 in conflict_pairs:
        if g1 in query_attrs:
            has_g1 = any(k in caption for k in attribute_groups[g1])
            has_g2 = any(k in caption for k in attribute_groups[g2])
            if has_g2 and not has_g1:
                boost *= penalty
    return boost
```

- apply_caption_filter : Extracts captions from Qdrant payloads, adjusts scores using _caption_filter_logic, and reranks the results

```python
def apply_caption_filter(query, candidates, penalty=0.5, boost_val=1.3):

    query_attrs, _, _ = extract_query_attrs(query)
    scored = []
    for id_, base_score in candidates:
        payload = qdrant_visual.retrieve(
            collection_name="visual", ids=[id_], with_payload=True
        )[0].payload
        caption = payload.get("caption", "")
        file_cat = payload.get("category", "")
        boost = _caption_filter_logic(
            caption, query_attrs, penalty, boost_val, file_cat=file_cat
        ) if caption else 1.0
        scored.append((id_, base_score * boost))
    scored.sort(key=lambda x: x[1], reverse=True)
    return scored
```

Implementation

Function Definitions

- rerank_with_kosroberta : Generates embeddings for the query and captions using ko-sroberta, calculates cosine similarity, and combines it with the visual score at a 0.5:0.5 ratio to determine the final ranking

```python
def rerank_with_kosroberta(query, candidates, top_n=10):
    """ko-sroberta로 쿼리-캡션 의미적 유사도 계산 후 재정렬"""
    query_emb = reranker.encode(query, normalize_embeddings=True)
    scored = []
    for id_, base_score in candidates:
        payload = qdrant_visual.retrieve(
            collection_name="visual", ids=[id_], with_payload=True
        )[0].payload
        caption = payload.get("caption", "")
        if caption:
            cap_emb = reranker.encode(caption, normalize_embeddings=True)
            sim = float(np.dot(query_emb, cap_emb))
            combined = base_score * 0.5 + sim * 0.5
        else:
            combined = base_score
        scored.append((id_, combined))
    scored.sort(key=lambda x: x[1], reverse=True)
    return scored[:top_n]
```

- display_results : Visually displays the search results. Retrieves the filename from the payload using visual_id. Generates a Cloudflare R2 URL by combining R2_BASE_URL with the filename

```python
def display_results(results, top_n=10):
    for rank, (id_, score) in enumerate(results[:top_n], 1):
        payload = qdrant_visual.retrieve(
            collection_name="visual", ids=[id_], with_payload=True
        )[0].payload
        filename = payload['filename']
        caption = payload.get("caption", "(캡션 없음)")
        url = f"https://pub-5966bf5d84f948c983500b6d9547eec9.r2.dev/masking_data/{filename}"
        print(f"#{rank} | {filename} | 점수: {score:.4f}")
        print(f"캡션: {caption}")
        display(IPImage(url=url, width=200))
        print()
```

Final Pipeline

Natural Language Query Input

Enter a natural language query

01

02

Query Preprocessing & Embedding

Fashion term substitution via fashion_dict → Google Translate
Embedding via marqo-fashionSigLIP text encoder
Category extraction, negation/positive expression extraction

Visual 검색 + Qdrant Payload Filter

Qdrant Visual Index Top-100 search (cosine similarity)
category filters, Filters positive and negative expressions
using must, and must_not conditions
Returns visual_id + payload(filename, caption, category)

03

Final Pipeline

Caption Keyword Filtering

References caption_category + micro_details + mood_and_tpo
attribute_groups boost / penalty
Penalty applied via conflict_pairs
Re-sort Top-50

04

05

ko-sroberta Reranking

Semantic similarity between Korean query ↔ Korean caption
Combine visual score + similarity
Return Top-10

06

Image Output

Output image via R2_BASE_URL + filename

Performance Comparison

- Easy: 2~3 simple attributes
  e.g., ribbed cropped knit top, turtleneck oversized knit
- Medium: 3~4 complex attributes + texture/silhouette expressions
  e.g., frayed wide denim pants, flowy floral midi dress
- Hard: mood + negation/positive + complex attributes + natural language
  e.g., feminine non-sheer blouse with shirring details,
  vintage-style checked wool coat with a belt

Difficulty	Final Model	Visual-Excluded Model
Easy	**0.9**	0.88
Medium	**0.78**	0.46
Hard	**0.86**	0.74
Average	**0.847**	0.693

Final Model: Visual + Caption Filtering + Reranking (Precision@10: 0.847)

By first retrieving visually similar candidates via image vector search, captures color, silhouette, texture, and natural language expressions not present in captions. Stable performance across all difficulty levels

Visual-Excluded Model: Text Filtering + Reranking (Precision@10: 0.693)

Captures keywords clearly present in captions, but fails on silhouette, texture, and natural language expressions not in captions

Pipeline Automation

Tool Introduction: Perfect

Why Automation?

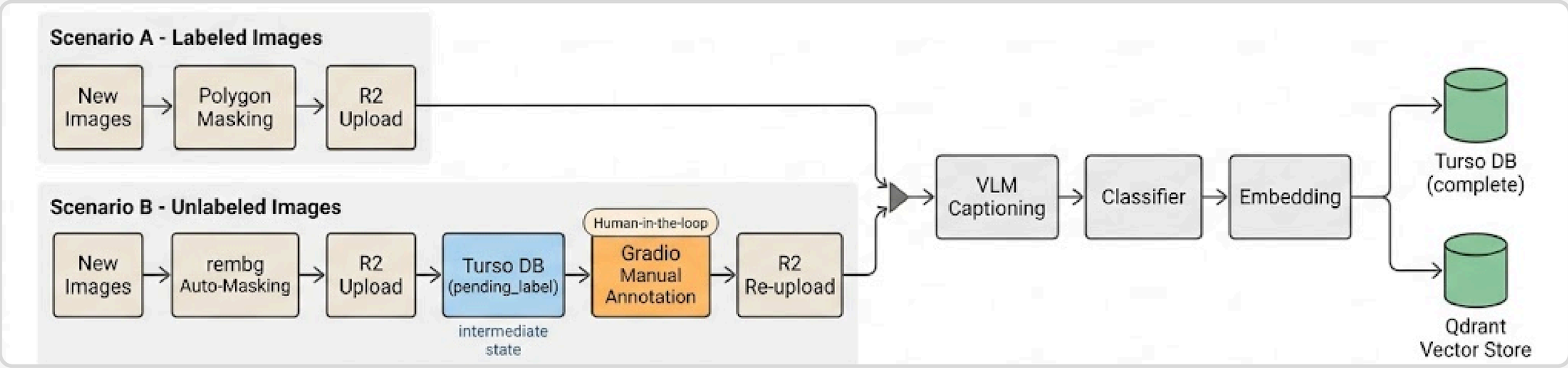
- Fashion data may arrive continuously — manual processing doesn't scale
- Inconsistent handling leads to gaps in DB / search index quality
- **In industry, automated pipelines are the standard for production ML systems**

What is Prefect?

- **Modern Python-native workflow orchestration framework (open-source)**
- Wraps Python functions with @flow / @task decorators — no YAML required
- Provides retry logic, logging, scheduling, and run visibility out of the box

Overall Pipeline Architecture

Trigger: New Images Arrive



Key Design Principle

- Both scenarios converge to the same final output: a fully indexed fashion item in Turso DB (status=complete) + Qdrant vector store
- Scenario B blocks at the Gradio step and resumes automatically once labeling is done

Live Demo & Limitations

Limitations

- Real-world scenarios may involve more **complex edge cases** (multi-item images, occlusion, etc.)
- Parts currently requiring manual input (Gradio annotation) could be further automated with better model training
- **Automated model retraining pipeline was not implemented** due to time constraints

Data Engineering Design Decisions

Parallelism & Batch Size Tuning

- VLM captioning: 70 concurrent requests via asyncio.Semaphore; R2 uploads: 16-worker ThreadPoolExecutor
- Batch sizes independently tuned per component: embeddings 64, Qdrant 1,000 pts, Turso 100 rows

Fault-Tolerant Pipeline Design

- Processed file_ids cached locally — zero DB SELECT inside processing loops
- Parses retryDelay from API error responses directly — respects API-advised backoff over fixed sleep

Minimizing API & DB Calls

- Processed file_ids cached locally — zero DB SELECT inside processing loops
- Parses retryDelay from API error responses directly — respects API-advised backoff over fixed sleep

Payload Index

- A Full-text index was created on the caption field to enable efficient keyword search within captions.
- A Keyword index was created on the category field to support fast exact-match filtering by category.

Thank You

Appendix
Manual Evaluation Results

Text Embedding

1st Evaluation

Method	Precision@10	Result
Visual only	**0.875**	Baseline
Visual + e5 RRF	**0.500** ↓	Rejected ✗
Visual + marqo RRF	**0.533** ↓	Rejected ✗
Visual + Caption Filtering	**0.950** ↑	Adopted ✓
Visual + marqo + Caption	**0.863**	Reference

Rejection Reasons

marqo text
Redundant with visual search — same model, same space

e5 multilingual
Fails color context: 'white blouse' → blouse with white stitching only

RRF fusion adds noise, not signal

2nd Evaluation

Method	Precision@10	Result
v1. Visual only	**0.671**	Baseline
v2. Marqo threshold	**0.438** ↓	Rejected ✗
v3. Category + Caption	**0.757** ↑	Adopted ✓
v4. micro_details only	**0.700** ↓	Rejected ✗

Rejection Reasons

v2. marqo threshold
Similarity threshold too sensitive and query-dependent — over-filters relevant candidates

v4. micro_details only
Lacks category context — no performance gain over v3

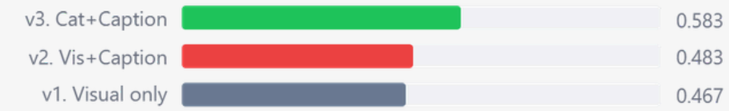
Caption filtering alone is limited without category pre-filtering

Each evaluation carried forward the top-performing method and the visual-only baseline as candidates for the next round.
Through evaluations 1 and 2, RRF-based fusion and threshold filtering were eliminated, leaving three finalists for next evaluation.

Manual Evaluation Results

3rd Evaluation — complex / hard

Method	Precision@10	Result
v1. Visual only	**0.467**	Baseline
v2. Visual + Caption	**0.483** ↓	Rejected ✗
v3. Category + Caption	**0.583** ↑	Adopted ✓



Rejection Reasons

v1. Visual only

Cannot handle negation, material/pattern confusion, or category noise

v2. Visual + Caption

Category noise unresolved — e.g. "shirt" query returns shirt-dresses

The final evaluation targeted queries that visual search alone struggles with
— composite conditions involving material, detail, and negation (e.g. "non-sheer, vertical stripe, white long dress").